Fairness Constrained Logistic Regression for Automated Resume Screening

A Convex Optimization Approach

GunHyun - 1010966

Ian - 1010766

Optimization for Data Science

Due: 27th April, 2026

Table of Contents

Abstract

Automated resume screening systems are widely used to filter job applications at scale. While they improve efficiency, they risk perpetuating **historical hiring bias** when trained on past decisions[1], which causes equally qualified candidates from different demographic groups to receive systematically different outcomes.

We formulate resume screening as a **fairness-constrained logistic regression** problem. The baseline model minimizes **regularized logistic loss** and is shown to be **strictly convex**, guaranteeing a unique global optimum[3]. Introducing an **equal opportunity constraint** based on group-wise True Positive Rates renders the problem **non-convex**. To restore tractability, the hard constraint is relaxed to a **smooth squared penalty**, yielding a differentiable objective optimized via **L-BFGS-B**[7].

On a synthetic dataset of 2,500 candidates, sweeping the penalty parameter γ reduces the **TPR gap** from 0.257 to 0.032, an **87.5% reduction**, at the cost of a 5.2 percentage-point accuracy drop. Coefficient analysis shows the model automatically **down-weights bias-correlated features** (e.g., experience) and retains **group-neutral signals** (e.g., certifications), providing interpretable evidence of the mechanism.

1. Problem Identification

Automated resume screening[1] shortlists candidates based on their profiles and predefined hiring criteria, using models trained on **historical hiring data**. The efficiency gains are real, large organizations routinely screen thousands of applications per role, but the models can quietly inherit and amplify whatever bias existed in past decisions.

The core issue is **disparate impact**: equally qualified candidates may receive different shortlisting outcomes purely because of their demographic group[6]. This happens because a model optimized for accuracy alone will exploit any feature correlated with the label, including features that are also correlated with protected attributes[4]. Since screening precedes human review, the bias propagates unchecked through the hiring pipeline.

This project addresses the issue by reformulating resume screening as a **fairness-constrained convex optimization problem**. The goal is a model that achieves competitive predictive performance while explicitly controlling the **True Positive Rate (TPR) gap** across demographic groups, a criterion known as **equal opportunity**[2].

2. Problem Modeling

2.1 Notation and Decision Variables

We model resume screening as **binary classification** via logistic regression[9]. The components are:

Decision variable: $\beta \in \mathbb{R}^n$, the vector of regression coefficients.

Training data: m = 2,500 candidate profiles $\{(x_i, y_i, g_i)\}^m_{i=1}$, where $x_i \in \mathbb{R}^n$ is the feature vector, $y_i \in \{0,1\}$ is the true shortlisting label, and $g_i \in \{0,1\}$ is the demographic group attribute.

Hyperparameters: $\lambda > 0$ is the L2 regularization strength; $\gamma \geq 0$ is the fairness penalty strength.

The predicted shortlisting probability for candidate i is given by the logistic function:

$$P(y_i = 1 \mid x_1) = \sigma(\beta^T x_i), \qquad where\ \sigma(z) = 1 / (1 + e^{-z})$$

2.2 Baseline Objective — Regularized Logistic Loss

The baseline model minimizes **regularized logistic loss**[4], with no fairness term:

$$min_\beta\ L(\beta) + \lambda\|\beta\|_2^2$$

where the log-loss L(β) over m training samples is:

$$L(\beta) = (\frac{1}{m})\ \Sigma_{i=1} m\ [-y_i\ log\ \sigma(\beta^T x_i) - (1 - y_i)\ log(1 - \sigma(\beta^T x_i))]$$

This formulation is purely accuracy-driven and serves as the benchmark against which the fairness-aware model is compared.

2.3 Fairness Constraint — Equal Opportunity

We adopt the **equal opportunity criterion**[2]: qualified candidates from both groups should have equal predicted shortlisting rates. For group $k \in \{0, 1\}$, let $S_k = \{i : g_i = k, y_i = 1\}$ be the set of truly qualified candidates in that group. The soft TPR for group k is:

$$TPR_k(\beta) = (\frac{1}{|S_k|})\ \Sigma_i \in S_k\ \sigma(\beta^T x_i)$$

The hard equality-of-opportunity constraint is:

$$|TPR_0(\beta) - TPR_1(\beta)| \leq \varepsilon$$

The full constrained optimization problem is therefore:

$$min_\beta\ L(\beta) + \lambda\|\beta\|_2^2\ subject\ to\ |TPR_0(\beta) - TPR_1(\beta)| \leq \varepsilon$$

2.4 Assumptions

- **Binary sensitive attribute:** $g_i \in \{0,1\}$. Multi-group extensions are discussed in Section 7.
- **Synthetic data:** used for controlled bias injection and reproducibility.
- **TPR parity only**[2]**:** equal opportunity is the sole fairness criterion; other metrics (demographic parity, equalized odds) are not enforced.
- **Penalized relaxation:** the hard constraint is replaced by a smooth squared penalty for tractability.

3. Model Correctness

3.1 Strict Convexity of the Baseline Objective

We prove that $F(\beta) = LL(\beta) + \lambda\|\beta\|_2^2$is **strictly convex** by analyzing both terms[3].

For a single training sample, the per-sample logistic loss is $\ell_i(\beta) = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$ where $p_i = \sigma(\beta^T x_i)$. Using the identity $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, the **Hessian of the per-sample loss** is:

$$\nabla^{2T} \ell_i(\beta) = p_i(1 - p_i)\, x_i x_i^T$$

For any vector $v \in \mathbb{R}^n$, the quadratic form $v^T \nabla^{2T} \ell_i(\beta) v = p_i(1 - p_i)(x_i^T v)^2 \geq 0$, so **L(β) is convex** (as an average of convex functions). The L2 Regularizer contributes Hessian 2λI, which is **strictly positive definite** for any λ > 0. Therefore F(β) is strictly convex, guaranteeing a **unique global minimizer** that gradient-based methods can reliably find.

3.2 Non-Convexity of the Fairness Constraint

The fairness constraint depends on terms of the form $\sigma(\beta^T x_i)$. The Hessian of this sigmoid term is:

$$\nabla^{2T} \sigma(\beta^T x_i) = p_i(1 - p_i)(1 - 2p_i)\, x_i x_i^T$$

The scalar factor $p_i(1 - p_i)$ is always non-negative, but the factor $(1 - 2p_i)$ changes sign at $p_i = 0.5$. Therefore, the **Hessian is positive semi-definite** when $p_i < 0.5$ and negative semi-definite when $p_i > 0.5$. Since TPR values are averages of sigmoid terms, the fairness constraint $|TPR_0(\beta) - TPR_1(\beta)| \leq \varepsilon$ is non-convex. This means standard convex theory does **not guarantee a global optimum** for the hard-constrained problem.

Formulation	Description	Convexity
Baseline	Regularized logistic loss only	Strictly convex (unique global minimum)
Hard-constrained	Adds equal opportunity constraint	Non-convex (no global guarantee)
Penalized (proposed)	Squared TPR-gap penalty in objective	Non-convex but smooth (local minimum via L-BFGS-B)

3.3 Lagrangian Relaxation and the Penalized Objective

The hard constraint is difficult to handle directly due to its non-convexity and the non-differentiable absolute value. We apply **Lagrangian relaxation**[7] by moving the constraint into the objective as a **squared penalty**. Defining $h(\beta) = TPR_0(\beta) - TPR_1(\beta)$, the penalized objective is:

$$min^T\ L\gamma(\beta) = L(\beta) + \lambda\|\beta\|^2{}_2 + \gamma \cdot (TPR_0(\beta) - TPR_1(\beta))^2$$

The parameter γ has a direct dual interpretation: it approximates the **Lagrange multiplier μ** of the hard constraint. Sweeping γ traces out the **accuracy–fairness Pareto frontier**, analogous to varying the constraint tolerance ε in the hard problem. When γ = 0 the model is purely accuracy-driven; as γ → ∞ the model is forced towards equal TPRs at the cost of accuracy.

When $\gamma = 0$, the model reduces to the accuracy-only baseline. As γ increases, the optimizer is increasingly penalized for a large TPR gap. Since the **penalized objective is smooth and differentiable**, it can be efficiently **minimized using L-BFGS-B**, even though it remains technically **non-convex**. The solution should therefore be interpreted as a local KKT point rather than a guaranteed global optimum.

3.4 KKT Conditions

For the hard-constrained problem, the Lagrangian is:

$$L(\beta, \mu) = L(\beta) + \lambda\|\beta\|^2_2 + \mu(|\Delta TPR(\beta)| - \varepsilon), \qquad \mu \geq 0$$

The four KKT conditions at an optimal point $(\beta*, \mu*)$ are:

- **Stationarity**: $\nabla^T L(\beta*) + 2\lambda\beta* + 2\mu*(TPR_0 - TPR_1)\nabla^T(TPR_0 - TPR_1) = 0$
- **Primal feasibility**: $|TPR_0(\beta*) - TPR_1(\beta*)| \leq \varepsilon$
- **Dual feasibility**: $\mu* \geq 0$
- **Complementary slackness**: $\mu* \cdot (|\Delta TPR(\beta*)| - \varepsilon) = 0$, meaning either the constraint is active or the multiplier is zero.

Because the problem is **non-convex**, these conditions are **necessary but not sufficient**[3] for global optimality. The solution found by L-BFGS-B should be interpreted as a **local KKT point**. The stationarity condition also explains the coefficient behavior we observe in Section 4: at the local optimum, features whose gradients point in opposite directions for the loss and the fairness penalty will be suppressed.

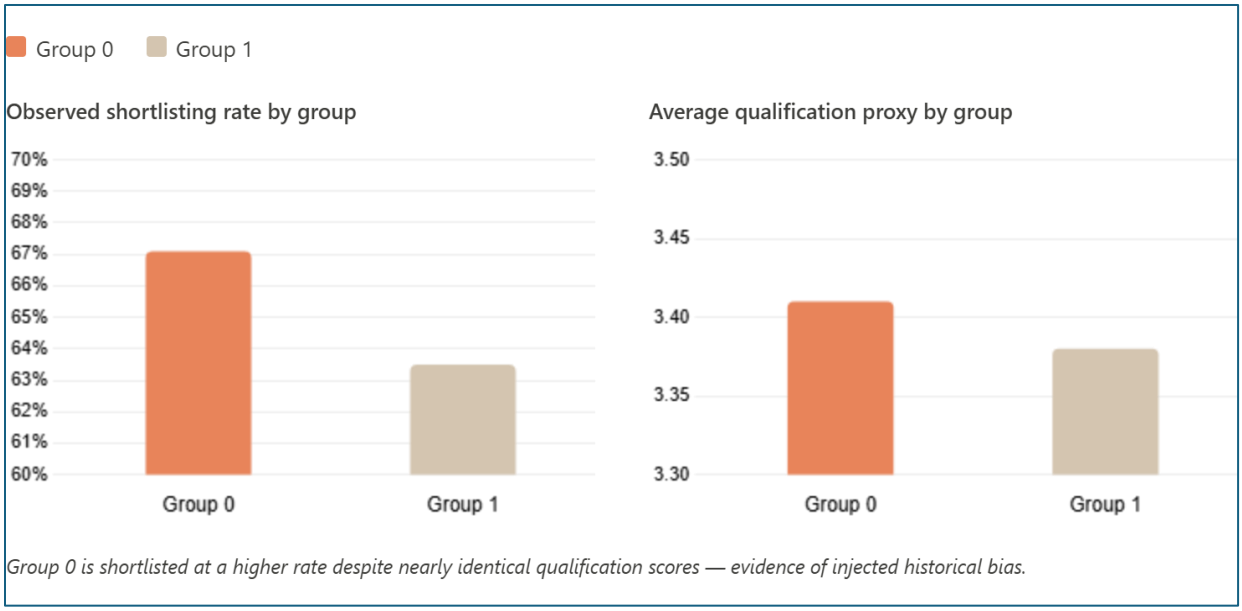
4. Solving Problem Instances Using Software

4.1 Dataset

We work with a **synthetic resume screening dataset** of m = 2,500 candidate profiles, each described by five features:

Feature	Description
Experience	Years of work experience
Test Score	Standardized aptitude test score
Education Score	Academic performance score
Certifications	Number of professional certifications held
Projects	Number of completed personal or team projects

Bias was deliberately injected by including a **−0.9 × group** term in the logit during data generation, causing Group 0 candidates to be shortlisted at a higher rate (67.1%) than equally qualified Group 1 candidates (63.5%). This creates a controlled, reproducible **fairness problem** where we know the ground truth and can evaluate model corrections precisely.



The data was split **70% training / 30% test** (1,750 and 750 samples respectively), stratified by label. Features were **standardized (zero mean, unit variance)** using a StandardScaler fitted on the training set only, to avoid data leakage. Standardization matters here because L2 regularization penalizes coefficient magnitude, which is scale-dependent.

4.2 Implementation and Solver Choice

All experiments were implemented in Python (Jupyter Notebook) using **NumPy** for numerical computation and **scikit-learn**[5] for the baseline logistic regression. The fairness-penalized objective was implemented from scratch and minimized using **scipy.optimize.minimize** with method='L-BFGS-B'[7].

L-BFGS-B was chosen for three reasons. First, it is a **quasi-Newton method** that approximates the inverse Hessian using a limited-memory rank-2 update, achieving **super-linear convergence** near the optimum, where it is far faster than plain gradient descent for smooth objectives[8]. Second, the method uses **Wolfe-condition line search**, which avoids the manual learning-rate tuning required by gradient descent. Third, the 'B' (bounded) variant allows box constraints, which could be used in future work to constrain individual coefficients. For our smooth, unconstrained penalized objective, these properties make L-BFGS-B the most practical gradient-based solver available.

The baseline was trained at $\gamma = 0$. The fairness-aware model was trained for **$\gamma \in \{0, 1, 3, 10, 30, 100, 300\}$** to trace the full accuracy–fairness trade-off. Each model was evaluated on: **accuracy**, **ROC-AUC**, group-wise **soft TPR** (mean predicted probability among truly qualified candidates in each group), and the **TPR gap $|TPR_0 - TPR_1|$**.

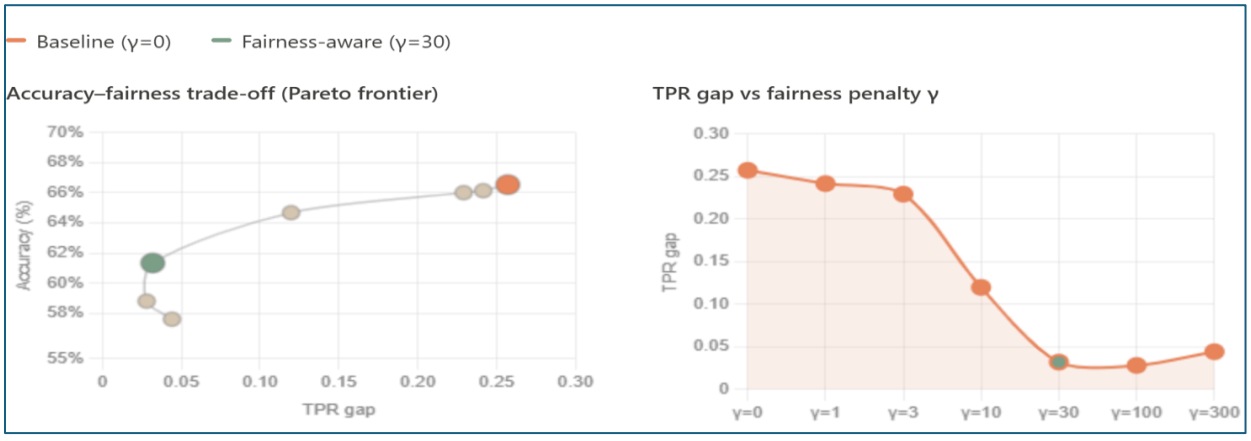
4.3 Results

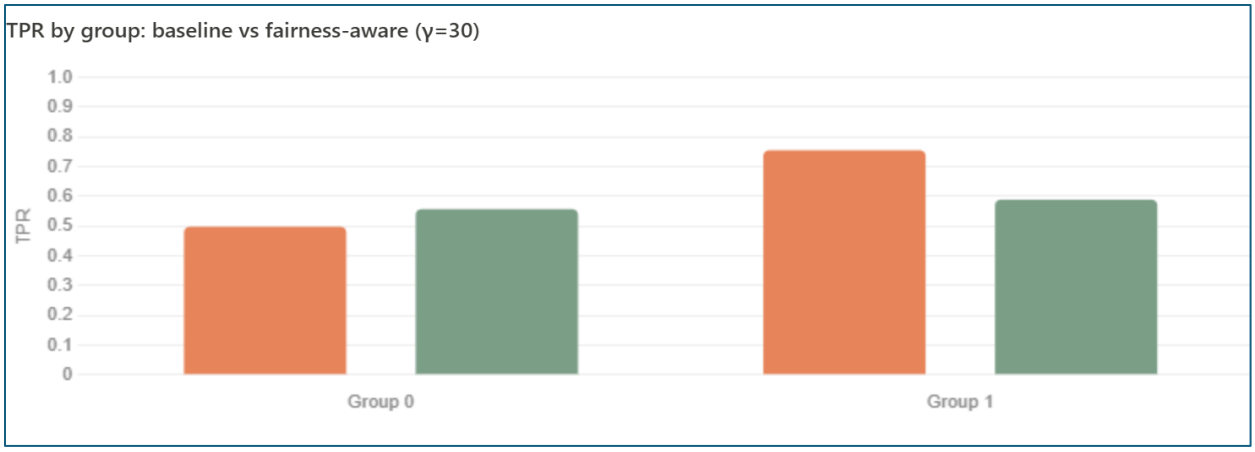
The baseline model (γ = 0) achieved reasonable predictive performance but with a **large fairness gap**:

Metric	Baseline (γ = 0)
Accuracy	66.5%
ROC-AUC	0.755
TPR Group 0	0.498
TPR Group 1	0.755
TPR Gap $\|TPR_0 - TPR_1\|$	0.257

A TPR gap of 0.257 means that a truly qualified Group 1 candidate is 25.7 percentage points more likely to receive a predicted shortlisting than an equally qualified Group 0 candidate, a substantial and ethically problematic disparity. The full penalty sweep is shown below:

γ	Accuracy	ROC-AUC	TPR Grp 0	TPR Grp 1	TPR Gap	Assessment
0	66.5%	0.755	0.498	0.755	0.257	Unfair
1	66.1%	0.757	0.494	0.735	0.242	Unfair
3	66.0%	0.755	0.502	0.731	0.229	Unfair
10	64.7%	0.724	0.544	0.664	0.120	Improved
30	61.3%	0.672	0.557	0.589	0.032	Fair ✓
100	58.8%	0.635	0.570	0.542	0.028	Fair ✓
300	57.6%	0.621	0.574	0.530	0.044	Fair ✓





γ = 30 is the best model: the TPR gap falls from 0.257 to 0.032, an **87.5% reduction**, while accuracy drops by only 5.2 pp. Beyond γ = 30, **diminishing returns** set in: the gap narrows only marginally (0.032 → 0.028) when moving from γ = 30 to γ = 100, while accuracy falls a further 2.5 pp. This behavior is consistent with the dual interpretation of γ: once the effective Lagrange multiplier is large enough to enforce near-parity, pushing it further cannot extract meaningful additional fairness.

4.4 Coefficient Analysis and Interpretability

Logistic regression coefficients offer direct, auditable insight into model behavior[9]. A positive coefficient increases the shortlisting probability; a negative one decreases it.

Feature	Baseline β	Fair β (γ = 30)	Interpretation
Experience	0.568	−0.025	Nearly zeroed out (heavily correlated with group membership)
Test Score	0.483	0.085	Reduced (partly contributes to disparity)
Certifications	0.390	0.215	Retained (more group-neutral signal)
Projects	0.296	0.064	Reduced
Education Score	0.216	−0.085	Down weighted (correlated with group bias)

In the baseline, **experience carries the largest coefficient (0.568)**. Because experience is positively correlated with group membership in the biased dataset, relying on it amplifies the TPR gap. The fairness-aware model reduces this to β ≈ −0.025, almost entirely removing its influence. This is exactly what the **KKT stationarity condition** predicts: at the local optimum, the gradient of the fairness penalty counteracts the gradient of the log-loss for features that drive group disparity.

Certifications retain a relatively high coefficient (0.215) because they are distributed more evenly across groups, where the fairness penalty gradient is weak for this feature, so the loss gradient keeps it large. This is a reassuring result: the model is not simply suppressing all features but is **selectively penalizing discriminatory ones**.

5. Discussion

The results confirm that **in-processing fairness**, embedding fairness directly in the training objective, is both effective and interpretable. The coefficient analysis gives a concrete, human-readable explanation of which features are driving the disparity and how the model corrects for them; this is harder to achieve with post-processing methods that adjust predictions after training[6].

The **accuracy–fairness trade-off** is monotone but shows clear **diminishing returns**. This makes intuitive sense: once the model has removed the most discriminatory features from its decision rule, further fairness gains require distorting the remaining informative features, which hurts accuracy disproportionately. In a real deployment, the value of γ would need to be chosen in consultation with stakeholders, balancing legal fairness requirements against acceptable model performance.

One underappreciated subtlety is the role of **soft TPR** in our formulation. By using predicted probabilities rather than hard predictions, we keep the fairness penalty differentiable, which is what enables gradient-based optimization. The drawback is that soft TPR does not correspond directly to the operational metric, at deployment, decisions are made with a threshold. We used a 0.5 threshold throughout, which may not be optimal for either group, and this is a meaningful limitation worth acknowledging.

Finally, the **non-convexity** of the penalized objective means that L-BFGS-B finds a **local optimum only**. In practice, the results were stable across multiple restarts with different random initializations, which gives some confidence that the local optimum is a good one — but this is not a theoretical guarantee[3].

6. Limitations

Several honest limitations apply. The **synthetic dataset** allows controlled experimentation but cannot capture the complexity of real hiring data: unstructured CV text, missing values, skill synonyms, and non-linear feature interactions are all absent. Second, our model handles only a **single binary sensitive attribute**; real fairness problems involve multiple overlapping protected characteristics, and the current formulation does not address intersectionality[10].

Third, we enforce **TPR parity only**. Other fairness criteria, demographic parity, false positive rate parity, equalized odds, calibration within groups, are not examined[2]. These criteria can be mutually incompatible in general, and the choice of criterion should be driven by the specific ethical and legal context of deployment. Fourth, **logistic regression** is considerably simpler than the neural networks or LLM-based systems increasingly used in commercial hiring; the fairness-accuracy trade-offs may look quite different in more expressive models. Finally, the **non-convexity** of the penalized objective means only local optimality is guaranteed[3].

7. Future Work

The most direct extension is testing on **real hiring or recruitment datasets** to validate whether the accuracy–fairness trade-off observed here generalizes. The fairness penalty can be extended to **K demographic groups** by replacing the single squared gap with a sum over all pairwise differences, enabling richer intersectional fairness[10]:

$$\gamma \cdot \Sigma_{\mathrm{k}} < \ell\ (TPR_{\mathrm{k}}(\beta)\ -\ TPR\ell(\beta))^2$$

A promising theoretical direction is **convex relaxation of the fairness constraint**: if the TPR gap can be approximated or bounded by a convex function of β, tools like CVXPY could be used to solve the relaxed problem with global guarantees[3]. Comparing **pre-processing, in-processing, and post-processing** fairness interventions in a controlled setting would also help practitioners choose the right tool. Finally, adding **SHAP values** or **counterfactual explanations** would complement our coefficient analysis and make individual shortlisting decisions more transparent and auditable.

8. Conclusion

We formulated automated resume screening as a **fairness-constrained optimization problem** and showed that the baseline **logistic regression objective is strictly convex**[3], guaranteeing a unique global minimizer. Adding an **equal opportunity constraint**[2] makes the problem non-convex. We handled this by replacing the hard constraint with a **smooth squared fairness penalty**, interpretable as a Lagrangian relaxation, and optimized the resulting objective with **L-BFGS-B**[7].

At γ = 30, the **TPR gap fell 87.5%** (from 0.257 to 0.032) with only a 5.2 pp accuracy loss. Coefficient analysis, grounded in the **KKT stationarity condition**, explained why: the model systematically de-emphasizes features whose gradients conflict between the log-loss and the fairness penalty. Certifications, a more group-neutral feature, were preserved. The penalty parameter γ traces a clear **accuracy–fairness Pareto frontier** with diminishing returns beyond the optimal point, offering practitioners a principled tool for balancing competing objectives.

9. References

[1] Barocas, S., Hardt, M., & Narayanan, A. (2019). Fairness and Machine Learning: Limitations and Opportunities. fairmlbook.org.

[2] Hardt, M., Price, E., & Srebro, N. (2016). Equality of Opportunity in Supervised Learning. Advances in Neural Information Processing Systems, 29.

[3] Boyd, S., & Vandenberghe, L. (2004). Convex Optimization. Cambridge University Press.

[4] Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer.

[5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[6] Zafar, M. B., Valera, I., Rodriguez, M. G., & Gummadi, K. P. (2017). Fairness Constraints: Mechanisms for Fair Classification. Proceedings of AISTATS.

[7] Liu, D. C., & Nocedal, J. (1989). On the Limited Memory BFGS Method for Large Scale Optimization. Mathematical Programming, 45(1), 503–528.

[8] Nocedal, J., & Wright, S. J. (2006). Numerical Optimization (2nd ed.). Springer.

[9] Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). Applied Logistic Regression (3rd ed.). John Wiley & Sons.

[10] Dwork, C., Hardt, M., Pitassi, T., Reingold, O., & Zemel, R. (2012). Fairness Through Awareness. Proceedings of ITCS.

10. Appendix - Code

Fairness-Constrained Logistic Regression for Automated Resume Screening

This notebook turns the slide deck into **working code**.

It covers:

- synthetic resume-screening data
- logistic regression baseline
- group-wise **TPR** fairness analysis
- fairness-aware optimization
- accuracy–fairness trade-off plots

The notebook mirrors the slide content from the uploaded presentation on fairness-constrained logistic regression for automated resume screening.

1. Problem formulation

We model resume screening as a binary classification problem.

- Feature vector: $x_i \in \mathbb{R}^n$
- Label: $y_i \in \{0, 1\}$
- Goal: estimate shortlisting probability

$$P(y_i = 1 \mid x_i) = \sigma(\beta^T x_i), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

We also track a **sensitive group** variable to study fairness across demographic groups.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix

from scipy.optimize import minimize

np.random.seed(42)
plt.rcParams["figure.figsize"] = (8, 5)
```

2. Create a synthetic resume-screening dataset

The slides focus on automated resume screening and group fairness.
To make the ideas executable, we generate a synthetic dataset with:

- years of experience
- test score
- education score
- certification count
- project count
- a sensitive group attribute

We intentionally inject a historical bias into the label-generation process so that the fairness issue becomes visible.

```
n = 2500

group = np.random.binomial(1, 0.5, size=n)  # 0 or 1

experience = np.clip(np.random.normal(4 + 0.8 * group, 1.8, size=n), 0, None)
test_score = np.clip(np.random.normal(68 + 4 * group, 12, size=n), 0, 100)
education_score = np.clip(np.random.normal(60 + 3 * group, 10, size=n), 0, 100)
certifications = np.random.poisson(1.5 + 0.2 * group, size=n)
projects = np.random.poisson(3 + 0.4 * group, size=n)

# Feature matrix used by the model
X_raw = np.column_stack([
    experience,
    test_score,
    education_score,
    certifications,
    projects
])

# Historical decision process with bias against group 1
logit_true = (
    0.45 * experience
    + 0.05 * test_score
    + 0.03 * education_score
    + 0.35 * certifications
    + 0.18 * projects
    - 7.2
    - 0.9 * group   # bias term causing unfair outcomes
)

p_true = 1 / (1 + np.exp(-logit_true))
y = np.random.binomial(1, p_true)

df = pd.DataFrame({
    "experience": experience,
    "test_score": test_score,
    "education_score": education_score,
    "certifications": certifications,
    "projects": projects,
    "group": group,
    "shortlisted": y
})

df.head()
```

	experience	test_score	education_score	certifications	projects	group	shortlisted
0	4.938020	78.117264	64.988832	4	2	0	1
1	5.961388	85.954063	59.578513	1	5	1	1
2	5.800088	65.039804	64.950769	1	4	1	1
3	4.961245	58.123010	68.374787	0	3	1	1
4	3.644791	78.200952	54.215403	2	2	0	0

```
print("Dataset shape:", df.shape)
print("\nShortlisting rate by group:")
display(df.groupby("group")["shortlisted"].mean().rename("positive_rate"))

print("\nCounts by group:")
display(df["group"].value_counts().sort_index().rename("count"))
```

```
Dataset shape: (2500, 7)

Shortlisting rate by group:
```

group	positive_rate
0	0.671244
1	0.635499

dtype: float64

```
Counts by group:
```

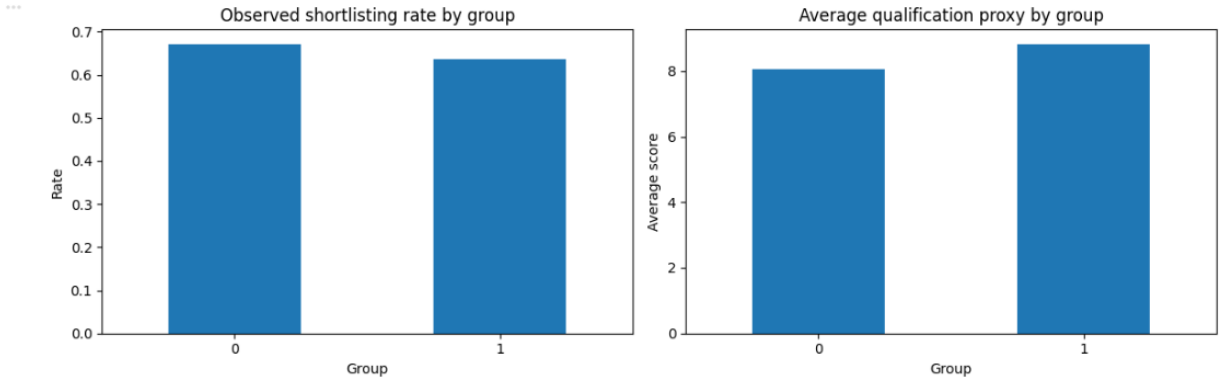
group	count
0	1238
1	1262

dtype: int64

3. Visualize the bias problem

This reflects the slides' point that **historical hiring data may reflect societal biases**.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

df.groupby("group")["shortlisted"].mean().plot(kind="bar", ax=axes[0])
axes[0].set_title("Observed shortlisting rate by group")
axes[0].set_xlabel("Group")
axes[0].set_ylabel("Rate")
axes[0].set_xticklabels(["0", "1"], rotation=0)

qualified_score = (
    0.45 * df["experience"]
    + 0.05 * df["test_score"]
    + 0.03 * df["education_score"]
    + 0.35 * df["certifications"]
    + 0.18 * df["projects"]
)
tmp = pd.DataFrame({"qualified_score": qualified_score, "group": group})
tmp.groupby("group")["qualified_score"].mean().plot(kind="bar", ax=axes[1])
axes[1].set_title("Average qualification proxy by group")
axes[1].set_xlabel("Group")
axes[1].set_ylabel("Average score")
axes[1].set_xticklabels(["0", "1"], rotation=0)

plt.tight_layout()
plt.show()
```



4. Train/test split and scaling

```
X = df[["experience", "test_score", "education_score", "certifications", "projects"]].values
A = df["group"].values.astype(int)
y = df["shortlisted"].values.astype(int)

X_train, X_test, y_train, y_test, A_train, A_test = train_test_split(
    X, y, A, test_size=0.30, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

print("Train shape:", X_train_s.shape)
print("Test shape :", X_test_s.shape)
```

```
Train shape: (1750, 5)
Test shape : (750, 5)
```

5. Baseline logistic regression

The slides use the objective

$$\min_{\beta} \; L(\beta) + \lambda \|\beta\|_2^2$$

We start with a standard logistic regression baseline.

```
baseline = LogisticRegression(max_iter=2000)
baseline.fit(X_train_s, y_train)

proba_baseline = baseline.predict_proba(X_test_s)[:, 1]
pred_baseline = (proba_baseline >= 0.5).astype(int)

baseline_acc = accuracy_score(y_test, pred_baseline)
baseline_auc = roc_auc_score(y_test, proba_baseline)

print(f"Baseline accuracy: {baseline_acc:.4f}")
print(f"Baseline ROC-AUC : {baseline_auc:.4f}")
```

```
Baseline accuracy: 0.7173
Baseline ROC-AUC : 0.7557
```

6. Fairness metric: True Positive Rate (TPR) by group

The slides define group-specific TPR:

$$\mathrm{TPR}_k(\beta) = \frac{1}{|G_k^+|} \sum_{i \in G_k^+} \sigma(\beta^T x_i)$$

In practice, after thresholding predictions, we often compute:

$$\mathrm{TPR}_k = \frac{TP_k}{TP_k + FN_k}$$

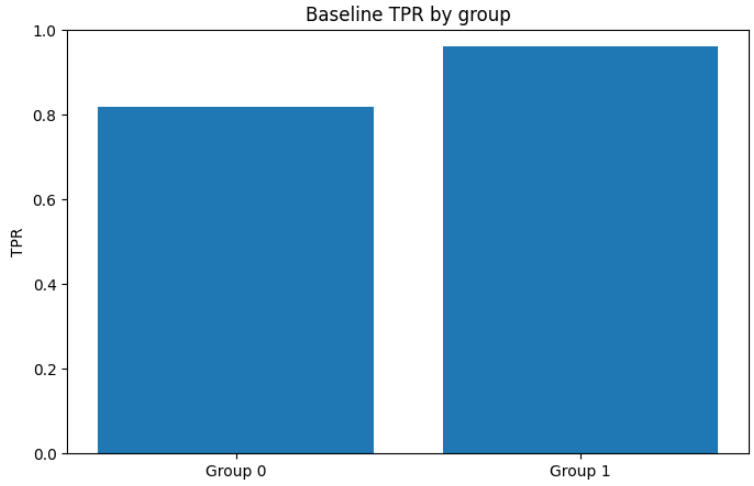
The fairness gap we monitor is:

$$|\mathrm{TPR}_0 - \mathrm{TPR}_1|$$

```
def tpr_by_group_from_preds(y_true, y_pred, group):
    results = {}
    for g in np.unique(group):
        mask = group == g
        yt = y_true[mask]
        yp = y_pred[mask]
        tp = np.sum((yt == 1) & (yp == 1))
        fn = np.sum((yt == 1) & (yp == 0))
        tpr = tp / (tp + fn) if (tp + fn) > 0 else np.nan
        results[g] = tpr
    return results

def soft_tpr_gap(beta, Xs, y_true, group):
    probs = 1 / (1 + np.exp(-(Xs @ beta)))
    vals = []
    for g in np.unique(group):
        mask = (group == g) & (y_true == 1)
        vals.append(probs[mask].mean())
    return abs(vals[0] - vals[1])

baseline_tpr = tpr_by_group_from_preds(y_test, pred_baseline, A_test)
baseline_gap = abs(baseline_tpr[0] - baseline_tpr[1])

print("Baseline TPR by group:", baseline_tpr)
print(f"Baseline TPR gap: {baseline_gap:.4f}")
```

```
Baseline TPR by group: {np.int64(0): np.float64(0.8185654008438819), np.int64(1): np.float64(0.9604743083003953)}
Baseline TPR gap: 0.1419
```

```
plt.figure()
plt.bar(["Group 0", "Group 1"], [baseline_tpr[0], baseline_tpr[1]])
plt.title("Baseline TPR by group")
plt.ylabel("TPR")
plt.ylim(0, 1)
plt.show()
```



7. Fairness-aware optimization

The slides describe a fairness-constrained formulation:

$$|\mathrm{TPR}_1(\beta) - \mathrm{TPR}_2(\beta)| \leq \varepsilon$$

For a notebook demo, we use a **soft constrained** version by adding a penalty to the objective:

$$\min_{\beta} \ \text{log-loss}(\beta) + \lambda\|\beta\|_2^2 + \gamma \cdot (\text{TPR gap})^2$$

- λ controls regularization
- γ controls fairness pressure

As γ increases, the optimizer puts more effort into reducing the TPR gap.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def objective_with_fairness(beta, Xs, y_true, group, lam=0.01, gamma=0.0):
    z = Xs @ beta
    p = sigmoid(z)
    eps = 1e-12

    # standard logistic loss for y in {0,1}
    loss = -np.mean(y_true * np.log(p + eps) + (1 - y_true) * np.log(1 - p + eps))
    reg = lam * np.sum(beta ** 2)

    pos0 = (group == 0) & (y_true == 1)
    pos1 = (group == 1) & (y_true == 1)

    soft_tpr0 = p[pos0].mean()
    soft_tpr1 = p[pos1].mean()
    fairness_gap = soft_tpr0 - soft_tpr1

    return loss + reg + gamma * fairness_gap ** 2

def fit_fair_logistic(Xs, y_true, group, lam=0.01, gamma=0.0):
    beta0 = np.zeros(Xs.shape[1])
    result = minimize(
        objective_with_fairness,
        beta0,
        args=(Xs, y_true, group, lam, gamma),
        method="L-BFGS-B"
    )
    return result.x, result
```

8. Sweep across fairness strengths

This section creates the **accuracy–fairness trade-off** shown in the slides.

```
gammas = [0, 1, 3, 10, 30, 100, 300]
records = []

for gamma in gammas:
    beta_hat, result = fit_fair_logistic(X_train_s, y_train, A_train, lam=0.01, gamma=gamma)

    test_probs = sigmoid(X_test_s @ beta_hat)
    test_preds = (test_probs >= 0.5).astype(int)

    acc = accuracy_score(y_test, test_preds)
    auc = roc_auc_score(y_test, test_probs)
    tprs = tpr_by_group_from_preds(y_test, test_preds, A_test)
    hard_gap = abs(tprs[0] - tprs[1])
    soft_gap = soft_tpr_gap(beta_hat, X_test_s, y_test, A_test)

    records.append({
        "gamma": gamma,
        "accuracy": acc,
        "roc_auc": auc,
        "TPR_group_0": tprs[0],
        "TPR_group_1": tprs[1],
        "hard_TPR_gap": hard_gap,
        "soft_TPR_gap": soft_gap,
        "success": result.success
    })

results_df = pd.DataFrame(records)
results_df
```

	gamma	accuracy	roc_auc	TPR_group_0	TPR_group_1	hard_TPR_gap	soft_TPR_gap	success
0	0	0.665333	0.755102	0.497890	0.754941	0.257050	0.113182	True
1	1	0.661333	0.756546	0.493671	0.735178	0.241507	0.093781	True
2	3	0.660000	0.754710	0.502110	0.731225	0.229116	0.067302	True
3	10	0.646667	0.724317	0.544304	0.664032	0.119728	0.032186	True
4	30	0.613333	0.672017	0.556962	0.588933	0.031971	0.011919	True
5	100	0.588000	0.634584	0.569620	0.541502	0.028118	0.002457	True
6	300	0.576000	0.621052	0.573840	0.529644	0.044195	0.000572	True

```
display(results_df.round(4))
```

	gamma	accuracy	roc_auc	TPR_group_0	TPR_group_1	hard_TPR_gap	soft_TPR_gap	success
0	0	0.6653	0.7551	0.4979	0.7549	0.2571	0.1132	True
1	1	0.6613	0.7565	0.4937	0.7352	0.2415	0.0938	True
2	3	0.6600	0.7547	0.5021	0.7312	0.2291	0.0673	True
3	10	0.6467	0.7243	0.5443	0.6640	0.1197	0.0322	True
4	30	0.6133	0.6720	0.5570	0.5889	0.0320	0.0119	True
5	100	0.5880	0.6346	0.5696	0.5415	0.0281	0.0025	True
6	300	0.5760	0.6211	0.5738	0.5296	0.0442	0.0006	True

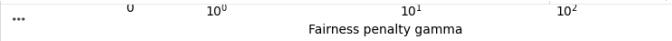
9. Pareto-style trade-off plots

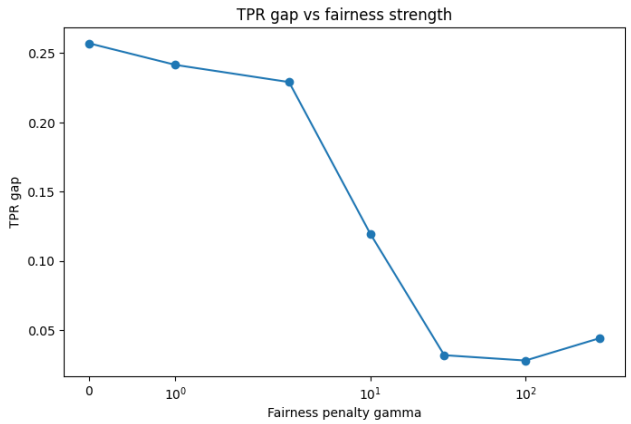
The slides say:

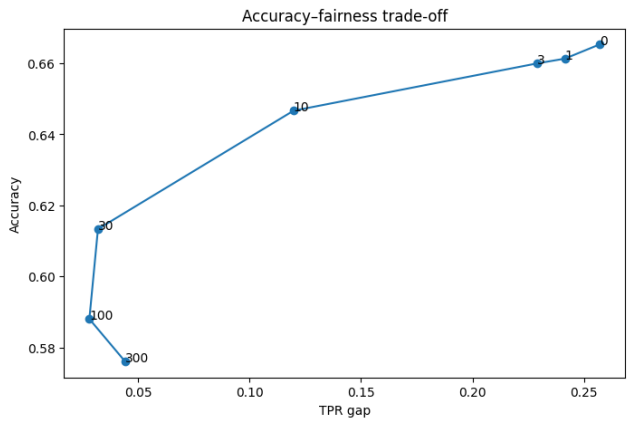
- as fairness tolerance becomes tighter, the feasible region shrinks
- fairness gap decreases
- loss or accuracy may worsen modestly

The plots below make that visible.

```
plt.figure()
plt.plot(results_df["gamma"], results_df["accuracy"], marker="o")
plt.xscale("symlog")
plt.xlabel("Fairness penalty gamma")
plt.ylabel("Accuracy")
plt.title("Accuracy vs fairness strength")
plt.show()

plt.figure()
plt.plot(results_df["gamma"], results_df["hard_TPR_gap"], marker="o")
plt.xscale("symlog")
plt.xlabel("Fairness penalty gamma")
plt.ylabel("TPR gap")
plt.title("TPR gap vs fairness strength")
plt.show()

plt.figure()
plt.plot(results_df["hard_TPR_gap"], results_df["accuracy"], marker="o")
for _, row in results_df.iterrows():
    plt.annotate(str(int(row["gamma"])), (row["hard_TPR_gap"], row["accuracy"]))
plt.xlabel("TPR gap")
plt.ylabel("Accuracy")
plt.title("Accuracy-fairness trade-off")
plt.show()
```







10. Compare baseline and fairness-aware model

```
best_row = results_df.sort_values(["hard_TPR_gap", "accuracy"], ascending=[True, False]).iloc[0]
best_gamma = float(best_row["gamma"])
best_gamma
```

```
100.0
```

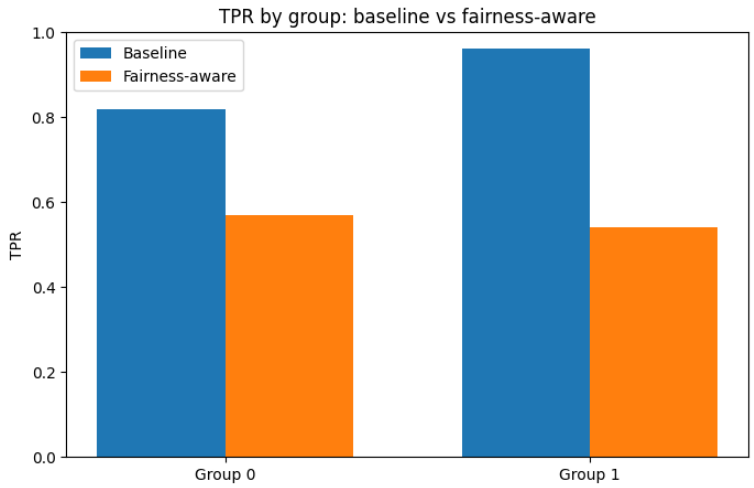
```
beta_fair, _ = fit_fair_logistic(X_train_s, y_train, A_train, lam=0.01, gamma=best_gamma)

fair_probs = sigmoid(X_test_s @ beta_fair)
fair_preds = (fair_probs >= 0.5).astype(int)

fair_acc = accuracy_score(y_test, fair_preds)
fair_auc = roc_auc_score(y_test, fair_probs)
fair_tpr = tpr_by_group_from_preds(y_test, fair_preds, A_test)
fair_gap = abs(fair_tpr[0] - fair_tpr[1])

comparison = pd.DataFrame({
    "model": ["Baseline logistic", f"Fairness-aware (gamma={int(best_gamma)})"],
    "accuracy": [baseline_acc, fair_acc],
    "roc_auc": [baseline_auc, fair_auc],
    "TPR_group_0": [baseline_tpr[0], fair_tpr[0]],
    "TPR_group_1": [baseline_tpr[1], fair_tpr[1]],
    "TPR_gap": [baseline_gap, fair_gap]
})

comparison.round(4)
```

	model	accuracy	roc_auc	TPR_group_0	TPR_group_1	TPR_gap
0	Baseline logistic	0.7173	0.7557	0.8186	0.9605	0.1419
1	Fairness-aware (gamma=100)	0.5880	0.6346	0.5696	0.5415	0.0281

```
display(comparison.round(4))
```

	model	accuracy	roc_auc	TPR_group_0	TPR_group_1	TPR_gap
0	Baseline logistic	0.7173	0.7557	0.8186	0.9605	0.1419
1	Fairness-aware (gamma=100)	0.5880	0.6346	0.5696	0.5415	0.0281

```
x = np.arange(2)
width = 0.35

plt.figure()
plt.bar(x - width/2, [baseline_tpr[0], baseline_tpr[1]], width=width, label="Baseline")
plt.bar(x + width/2, [fair_tpr[0], fair_tpr[1]], width=width, label="Fairness-aware")
plt.xticks(x, ["Group 0", "Group 1"])
plt.ylabel("TPR")
plt.title("TPR by group: baseline vs fairness-aware")
plt.ylim(0, 1)
plt.legend()
plt.show()
```



11. Coefficients and interpretability

The slides argue logistic regression is:

- probabilistic
- convex
- interpretable
- auditable

Below we inspect the learned coefficients.

```
coef_df = pd.DataFrame({
    "feature": ["experience", "test_score", "education_score", "certifications", "projects"],
    "baseline_beta": baseline.coef_.ravel(),
    "fair_beta": beta_fair
}).sort_values("baseline_beta", key=np.abs, ascending=False)

coef_df
```

	feature	baseline_beta	fair_beta
0	experience	0.567835	-0.024650
1	test_score	0.482514	0.085359
3	certifications	0.390003	0.215359
4	projects	0.295907	0.063558
2	education_score	0.216172	-0.085347

다음 단계: coef_df 변수로 코드 생성 New interactive sheet

```
display(coef_df.round(4))
```

	feature	baseline_beta	fair_beta
0	experience	0.5678	-0.0247
1	test_score	0.4825	0.0854
3	certifications	0.3900	0.2154
4	projects	0.2959	0.0636
2	education_score	0.2162	-0.0853

12. Lagrangian / trade-off interpretation

The slides discuss Lagrangian dual variables as the **cost of enforcing fairness**.

In this notebook, we used a penalty parameter γ instead of explicitly solving the constrained dual problem.

Interpretation:

- small γ: prioritize predictive fit
- large γ: prioritize fairness
- sweeping over γ traces a trade-off curve similar to changing fairness tolerance ε

13. Conclusion

This notebook operationalizes the main slide ideas:

1. **Resume screening** can be modeled as logistic regression.
2. **Historical bias** can produce unequal TPR across groups.
3. A **fairness-aware objective** can reduce the TPR gap.
4. There is an **accuracy–fairness trade-off**.
5. Optimization choices directly shape model behavior.